



Módulo 3 Fundamentos de animación 2D

Preparado por: Ing. Samuel Jiménez



Unidad 4

Eventos del Teclado, Materiales e iluminación

Objetivos Generales

- ***Describir los modelos de sombreados para dale color a una superficie.***
- ***Mostrar el uso de los comandos en OpenGL para la definición de las propiedades de material de un objeto.***



Gestión a los cambios de eventos de ventana

Cuando se desea redimensionar la ventana que se está ejecutando solo basta con pulsar el ratón en cualquier esquina y desplazarlo.

Esta acción genera un redimensionamiento del evento al cual el sistema puede responder.

La función **glutReshapeFunc** es la encargada de establecer la función de devolución de llamada de un cambio de forma para la ventana actual.

Sintaxis: `void glutReshapeFunc (void(*fun)(int ancho, int alto))`

Gestión a los cambios de eventos de ratón

La función para atender y tratar los eventos generados por el usuario mediante el ratón, tales como pulsar y soltar botones y la acción de arrastrares:

Void glutMouseFunc(void (*func)(int botón, int estado, int x, int y))

Botón: en el parámetro botón existen tres valores válidos: GLUT_LEFT_BUTTON, GLUT_MIDDLE_BUTTON, GLUT_RIGHT_BUTTON.

Estado: el parámetro de estado puede ser GLUT_UP o GLUT_DOWN, cuando el usuario suelta un botón o lo presiona.

x e y: Posición del cursor en la ventana

Gestión a los cambios de eventos de teclado

La función para atender los eventos del teclado es :

Void glutKeyboardFunc (void (*func)(unsigned char tecla, intx, inty))

Tecla: indica el código ASCII de la tecla pulsada

x, y: Posición del cursor en la ventana en el momento de producirse la pulsación

Func: nombre de la función que va a llamar Glut cuando se produzca una pulsación de tecla.

Gestión a los cambios de eventos de teclado

GLUT_KEY_F1	F1
GLUT_KEY_F2	F2
GLUT_KEY_F3	F3
GLUT_KEY_F4	F4
GLUT_KEY_F5	F5
GLUT_KEY_F6	F6
GLUT_KEY_F7	F7
GLUT_KEY_F8	F8

Gestión a los cambios de eventos de teclado

GLUT_KEY_F9	F9
GLUT_KEY_F10	F10
GLUT_KEY_F11	F11
GLUT_KEY_F12	F12
GLUT_KEY_LEFT	Left
GLUT_KEY_RIGHT	Right
GLUT_KEY_UP	Up
GLUT_KEY_DOWN	Down

Gestión a los cambios de eventos de teclado

GLUT_KEY_PAGE_UP	Up
GLUT_KEY_PAGE_DOWN	Down
GLUT_KEY_HOME	Home
GLUT_KEY_END	End
GLUT_KEY_INSERT	Insert

Especificación de un normal en OpenGL

La especificación de los normales en OpenGL es empleada para la iluminación, en los que se indica la orientación referente de un objeto respecto a las fuentes de iluminación.

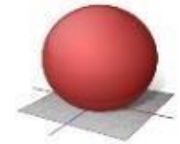
Sintaxis:

Void glNormal3f(TIPO nx, TIPO ny, TIPO nz)

nx=especifica la magnitud x del vector normal

ny=especifica la magnitud y del vector normal

nz=especifica la magnitud z del vector normal



```
glBegin(GL_QUADS);  
glNormal3f(0, 1, 0);  
  
glVertex3f(0, 0, 0);  
glVertex3f(3, 0, 0);  
glVertex3f(3, 3, 0);  
glVertex3f(0, 3, 0);  
glEnd();
```

Cuando se establece la normal, cada uno de los vértices que se dibujen a continuación asumirá la misma especificación. Si esto no es lo que se requiere, entonces se debe especificar de forma explícita una normal para cada uno de los vértices a dibujar.

Modelo de sombreado

Para darle color a una superficie mediante OpenGL, se puede rellenar con un color sólido o con una combinación de los colores de los vértices.

Estas son:

- Sombreado constante: utilizando un solo color para toda la línea o polígono
- Sombreado suave: Utilizando diferentes colores para toda la línea o polígono

OpenGL permite elegir entre dos métodos de sombreado, de caras planas y suaves.

La función que establece el sombreado predeterminado en liso o en suave es la función **glShadeModel**.

Sintaxis: void glShadeModel (GLenum mode)

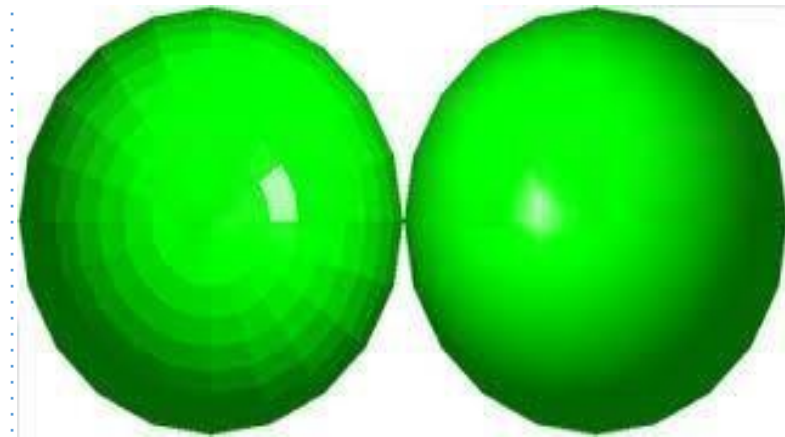
Modelo de sombreado

Sintaxis: void glShadeModel (GLenum mode)

El parámetro mode indica el tipo de sombreado **GL_FLAT** (liso o constante) y **GL_SMOOTH** (suave)

GL_FLAT: la superficie se rellena con el color
Sólido seleccionado

GL_SMOOTH: La superficie se rellena con una
Progresión de colores calculada a partir de los
colores de los vértices.



Ejemplo de Modelo de sombreado

```
glShadeModel(GL_FLAT);  
glBegin(GL_TRIANGLE_STRIP);  
    glColor3( 0.0, 0.5, 0.0 );  
    glVertex3( -1.0, -0.5, -5.0 );  
  
    glColor3( 1.0, 0.0, 0.0 );  
    glVertex3( 1.0, -0.5, -5.0 );  
  
    glColor3( 1.0, 0.0, 0.0 );  
    glVertex3( 0.0, 0.5, -5.0 );  
  
    glColor3( 1.0, 0.0, 0.0 );  
    glVertex3( 1.5, 0.0, -5.0 );  
  
    glColor3( 0.0, 0.0, 1.0 );  
    glVertex3( 2.0, -1.5, -5.0 );  
glEnd();
```



Modelo de sombreado

En OpenGL la función **glEnable (GL_DEPTH_TEST)** es la encargada de proporcionar el efecto de profundidad cuando se renderizan las figuras.

Algo muy importante a considerar es que si un objeto tapa a otro este efecto no se vera.

Además, se debe usar la función `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)`

Propiedad del material

OpenGL dispone de dos funciones que se utilizan para especificar las propiedades que caracterizan el material de un determinado objeto en el mundo real, estas funciones son:

Void glMaterial {if}(GLenum fase, GLenum pname, TYPEparam)

Donde

glMaterialf: se usa para las propiedades que requieren un parámetro escalar

glMaterialfv: se usa para las propiedades que requieren un vector.

face: especifica si se están estableciendo las propiedades materiales frontales, posteriores o ambas.

Los valores pueden ser **GL_FRONT, GL_BACK y GL_FRONT_AND_BACK**

pname: especifica el parámetro de material de un solo valor estableciendo para las dos primeras variantes. Los valores son **GL_AMBIENT, GL_DIFFUSE, GL_AMBIENT_AND_DIFFUSE, GL_SPECULAR, GL_EMISSION, GL_COLOR_INDEXES, GL_SHININESS.**

Propiedad del material

GL_AMBIENT:Color ambiente del material. Depende de las fuentes de luz y de suposición relativa al objeto. Por defecto es (0.2,0.2,0.2,1.0) para ambas caras.

GL_AMBIENT_AND_DIFFUSE: Color ambiente y difuso del material.

GL_COLOR_INDEXES: índice de color ambiente, difuso y especular.

GL_DIFFUSE:Color difuso del material. Por defecto es (0.8,0.8,0.8,1.0) para ambas caras.

GL_EMISSION:Color emisivo del material. Por defecto es (0.0,0.0,0.0,1.0) para ambas caras.

GL_SHININESS: Exponente especular.

GL_SPECULAR:Color especular del material. Por defecto es (0.0,0.0,0.0,1.0) para ambas caras.

Definición de las características de los objetos básicos en OpenGL

Material	Color ambiente	Color difuso	Color especular	Brillo
Aluminio	0.30 0.30 0.35	0.30 0.30 0.50	0.70 0.70 0.80	9
Azul	0.10 0.11 0.79	0.30 0.30 0.71	0.83 0.83 0.83	12
Cobre	0.33 0.26 0.23	0.50 0.11 0.00	0.95 0.73 0.00	93
Oro	0.57 0.40 0.00	0.22 0.15 0.00	0.71 0.70 0.56	16
Púrpura	0.25 0.17 0.19	0.10 0.03 0.22	0.64 0.00 0.98	8
Rojo	0.25 0.15 0.15	0.27 0.00 0.00	0.61 0.13 0.18	12

Cambios de las propiedades del Material

Para realizar el cambio de las propiedades del material utilizamos la función `glColorMaterial`.

Sintaxis: `void glColorMaterial (GLenum cara, GLenum modo)`

Cara: especifica si deben seguir **GL_FRONT**, **GL_BACK** o **GL_FRONT_AND_BACK**, al color actual.

Modo: especifica las propiedades materiales que deben seguir al color actual. Estas son **GL_EMISSION**, **GL_SPECULAR**, **GL_AMBIENT**, **GL_DIFFUSE** o **GL_AMBIENT_AND_DIFFUSE**.

Cambios de las propiedades del Material

Podemos cambiar el color ahora para la nueva cara utilizando glColor. Después de realizar estos cambios debemos llamar **glEnable** con el argumento **GL_COLOR_MATERIAL** para que los cambios tengan el efecto deseado.

```
glColorMaterial(GL_FRONT, GL_DIFFUSE)  
glEnable(GL_COLOR_MATERIAL)
```

Para desactivarla se emplea la orden: **glDisable(GL_COLOR_MATERIAL)**

Luz

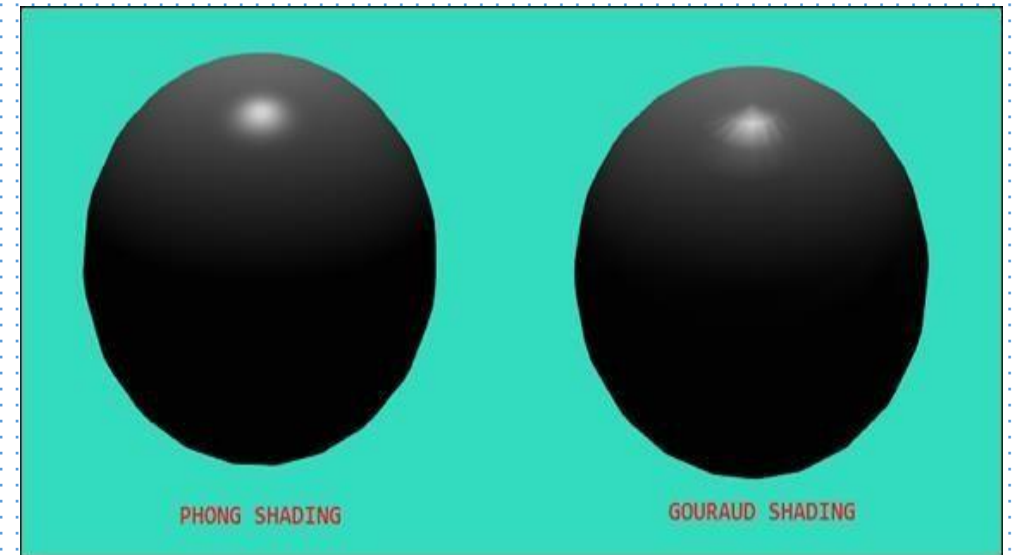
Ambiente La luz ambiente es aquella luz que no tiene origen concreto.

La luz ambiente se produce al rebotar la luz sobre varias superficies del entorno, por lo que es casi imperceptible en exteriores, y mas relevantes en interiores.

En OpenGL la luz ambiente de una escena se especifica dando sus componentes roja, verde y azul. Para ello se utiliza la siguiente orden:

glLightModel*{f}*v (Glenum nompar, TIPO*param)

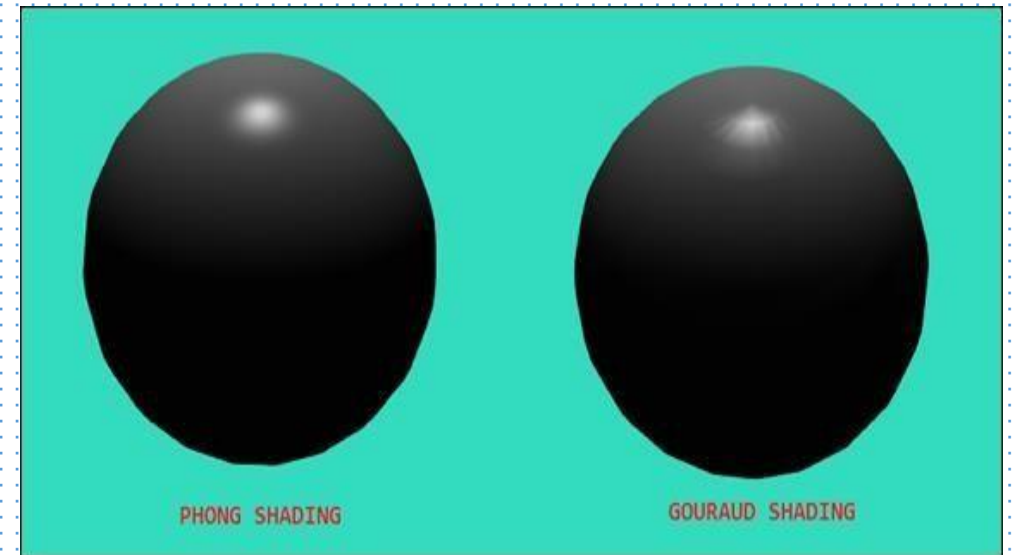
Nompar: especifica un parámetro de modelo de iluminación



Luz Ambiente

glLightModel{f}v (GLenum nompar, TPO* param)

Nompar acepta: GL_LIGHT_MODEL_AMBIENT,
GL_LIGHT_MODEL_LOCAL_VIEWER,
GL_LIGHT_MODEL_TWO_SIDE,
GL_LIGHT_MODEL_COLOR_CONTROL



Luz Posicional

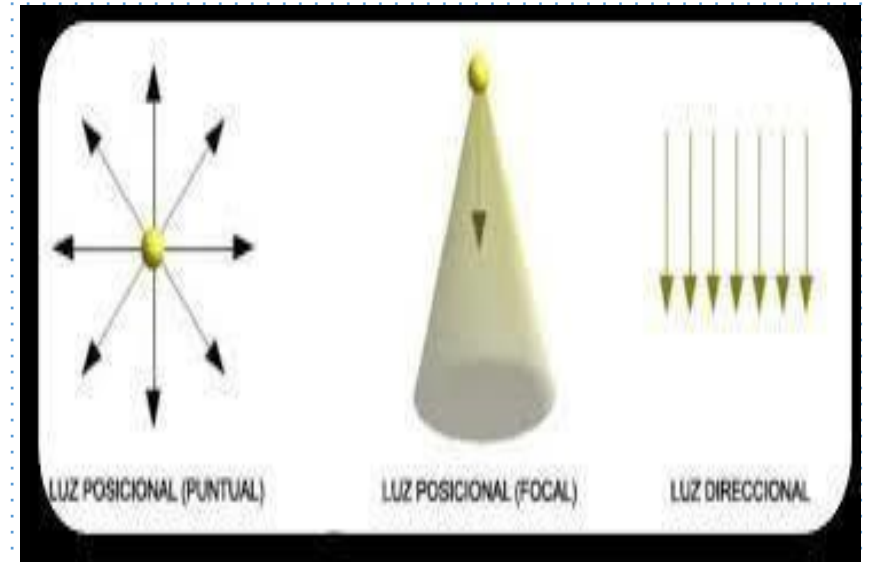
El efecto de las luces posicionales es como los bombillos que radian luz en todas las direcciones desde una localización puntual.

La luz posicional sirve para situar puntos de luz en una escena.

Para crear una fuente de luz posicional en OpenGL se usa la orden `glLight()`

Formato:

`glLight {f}(GLenum fuente, GLenum nompar, TIPO param)`



Luz Posicional

Formato:

`glLight{f}(GLenum fuente, GLenum nompar, TIPO param)`

Fuente: especifica que origen de luz se esta modificando

nompar: especifica que parámetro de iluminación se establecerá mediante esta llamada de función.

Nompar puede ser:

- `GL_AMBIENT`: Intensidad RGBA ambiente de la fuente
- `GL_CONSTANT_ATTENUATION`: Atenuación constante
- `GL_DIFFUSE`: Intensidad RGBA difusa de la fuente
- `GL_LINEAR_ATTENUATION`: Atenuación lineal
- `GL_POSITION`: Posición del origen de la fuente
- `GL_QUADRATIC_ATTENUATION`: Atenuación cuadrática

Luz Direccional

El efecto de las luces direccionales es la de emitir rayos de luz paralelos, cada uno con la misma dirección.

En OpenGL, las fuentes de luz direccionales se especifican mediante la orden `glLight*()`

Formato: `glLight {if}(GLenum fuente, GLenum nompar, TIPO* param)`

Gestión de foco de luz

Cuando se coloca un foco de luz en una escena, lo que se observa en esa escena es la luz que llega hasta el punto en el que se encuentra la cámara.

Si una escena no contara con una fuente de luz, la escena se convertiría en una imagen negra.

Si una escena no contara con una fuente de luz, la escena se convertiría en una imagen negra.

OpenGL ofrece 4 comandos de luces, dos escalares y dos vectoriales. Los dos comandos de gestión de luces escalares son:

glLightf(Glenum luz, Glenum atributo, GLfloat valor)

glLighti(Glenum luz, Glenum atributo, GLfloat valor)

Gestión de foco de luz

glLightf(GLenum luz, GLenum atributo, GLfloat valor)

glLighti(GLenum luz, GLenum atributo, GLint valor)

Luz: Especifica el número de la luz con la que se está trabajando

Atributo: Especifica el parámetro de la luz que se desea alterar

Valor: Especifica el valor que sustituirá al antiguo en el atributo

Los atributos permitidos son:

GL_SPOT_CUTOFF: Especifica el máximo ángulo de apertura de la fuente de luz en el rango [0,90] y el valor especial 180.

GL_SPOT_EXPONENT: Especifica la distribución de intensidad de luz en el rango [0,128]

Gestión de la atenuación de la intensidad de luz

La gestión de la atenuación de la luz en OpenGL se refiere al efecto que se producen cuando la intensidad de la luz se disminuye al propagarse desde donde se produce hasta el lugar desde el que se observa.

OpenGL dispone de tres tipos diferentes de atenuación: constante (c) , Lineal (L), cuadrática (Q)

La función OpenGL que se usa para la gestión de la atenuación de las luces esta se basa en las funciones

glLightf(Glenum luz, Glenum atributo, GLfloat valor)

glLighti(Glenum luz, Glenum atributo, GLfloat valor)

Gestión de la atenuación de la intensidad de luz

glLightf(Glenum luz, Glenum atributo, GLfloat valor)

glLighti(Glenum luz, Glenum atributo, GLfloat valor)

Luz: especifica el número de la luz que se le cambia a la atenuación

Atributo: Puede contener cualquiera de los siguientes valores tipados:

GL_CONSTANT_ATTENUATION, GL_LINEAR_ATTENUATION o GL_QUADRATIC_ATTENUATION

Valor: especifica numéricamente el valor de atenuación elegido y es siempre positivo.

Ejemplo de luz direccional

```
#include <windows.h>
#include <windows.h>
#include <gl/glut.h>
void iniciar()
{
    GLfloat color_esfera_ambiente[]={0.5,0.5,0.5,1.0};
    GLfloat color_esfera_difuso[]={0.5,0.5,0.5,1.0};
    GLfloat color_esfera_especular[]={0.5,0.5,0.5,1.0};
    GLfloat brillo_esfera_especular[]={5.0};
    GLfloat luz_ambiente[]={0.2,0.2,0.2,1.0};
    glClearColor(0.0,0.0,0.0,0.0);
    glMaterialfv(GL_FRONT, GL_AMBIENT, color_esfera_ambiente);
    glMaterialfv(GL_FRONT, GL_DIFFUSE, color_esfera_difuso);
    glMaterialfv(GL_FRONT, GL_SPECULAR, color_esfera_especular);
    glMaterialfv(GL_FRONT, GL_SHININESS, brillo_esfera_especular);
    glLightModelfv(GL_LIGHT_MODEL_AMBIENT, luz_ambiente);
    glEnable(GL_LIGHTING);
    glEnable(GL_LIGHT0);
}
```

```
glPopMatrix();
glPushMatrix();
glTranslatef(0.7,0.0,0.0);
glLightfv(GL_LIGHT0, GL_AMBIENT, luz_verde_ambiente);
glLightfv(GL_LIGHT0, GL_DIFFUSE, luz_verde_difusa);
glLightfv(GL_LIGHT0, GL_SPECULAR, luz_verde_especular);
glLightfv(GL_LIGHT0, GL_POSITION, luz_direccion_oblicua);
glutSolidSphere(0.3, 50, 50);
glPopMatrix();
glFlush();
}
```

```
void dibuja()
{
    GLfloat luz_blanca_ambiente[]={0.2,0.2,0.2,1.0};
    GLfloat luz_blanca_difusa[]={0.8,0.8,0.8,1.0};
    GLfloat luz_blanca_especular[]={0.5,0.5,0.5,1.0};
    GLfloat luz_verde_ambiente[]={0.2,0.5,0.2,1.0};
    GLfloat luz_verde_difusa[]={0.3,0.8,0.3,1.0};
    GLfloat luz_verde_especular[]={0.2,0.5,0.2,1.0};
    GLfloat luz_direccion_zenit[]={0.0,1.0,0.0,0.0};
    GLfloat luz_direccion_oblicua[]={1.0,1.0,0.0,0.0};
    glClear(GL_COLOR_BUFFER_BIT);
}
```

```
int main (int argc, char**argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB );
    glutInitWindowPosition(100,100);
    glutInitWindowSize(500,500);
    glutCreateWindow("Luz Direccional");
    iniciar();
    glutDisplayFunc(dibuja);
    glutMainLoop();
    return 0;
}
```

Ejemplo de luz posicional

```
#include <windows.h>
#include <gl/glut.h>
void iniciar(void)
{
    GLfloat color_esfera_ambiente[]={0.5,0.5,0.5,1.0};
    GLfloat color_esfera_difuso[]={0.8,0.8,0.8,1.0};
    GLfloat color_esfera_especular[]={0.5,0.5,0.5,1.0};
    GLfloat brillo_esfera_especular[]={5.0};
    GLfloat luz_ambiente[]={0.9,0.9,0.2,1.0};
    glMaterialfv(GL_FRONT, GL_AMBIENT, color_esfera_ambiente);
    glMaterialfv(GL_FRONT, GL_DIFFUSE, color_esfera_difuso);
    glMaterialfv(GL_FRONT, GL_SPECULAR, color_esfera_especular);
    glMaterialfv(GL_FRONT, GL_SHININESS, brillo_esfera_especular);
    glLightModelfv(GL_LIGHT_MODEL_AMBIENT, luz_ambiente);
    glEnable(GL_LIGHTING);
    glEnable(GL_LIGHT0);
    glClearColor(0.0,0.0,0.0,0.0);
}
```

```
int main (int argc, char**argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB );
    glutInitWindowPosition(100,100);
    glutInitWindowSize(500,500);
    glutCreateWindow("Luz Posicional");
    iniciar();
    glutDisplayFunc(dibuja);
    glutMainLoop();
    return 0;
}
```

```
void dibuja(void)
{
    GLfloat luz_blanca_ambiente[]={0.1,0.1,0.1,1.0};
    GLfloat luz_blanca_difusa[]={0.8,0.8,0.8,1.0};
    GLfloat luz_blanca_especular[]={0.4,0.4,0.4,1.0};
    GLfloat luz_roja_ambiente[]={0.2,0.0,0.0,1.0};
    GLfloat luz_roja_difusa[]={0.8,0.3,0.3,1.0};
    GLfloat luz_roja_especular[]={0.4,0.1,0.1,1.0};
    GLfloat luz_posicion_baja[]={0.0,0.0,-0.5,1.0};
    GLfloat luz_posicion_alta[]={0.0,0.2,-0.8,1.0};
    glClear(GL_COLOR_BUFFER_BIT);
    glLightfv(GL_LIGHT0, GL_AMBIENT, luz_roja_ambiente);
    glLightfv(GL_LIGHT0, GL_DIFFUSE, luz_roja_difusa);
    glLightfv(GL_LIGHT0, GL_SPECULAR, luz_roja_especular);
    glLightfv(GL_LIGHT0, GL_POSITION, luz_posicion_baja);

    glPopMatrix();
    glPushMatrix();
    glShadeModel(GL_FLAT);
    glTranslatef(0.5,0.0,0.0);
    glLightf(GL_LIGHT0, GL_CONSTANT_ATTENUATION, 0.0);
    glLightf(GL_LIGHT0, GL_LINEAR_ATTENUATION, 1.0);
    glLightf(GL_LIGHT0, GL_QUADRATIC_ATTENUATION, 0.0);
    glLightfv(GL_LIGHT0, GL_POSITION, luz_posicion_alta);
    glutSolidSphere(0.2,20,20);
    glPopMatrix();
    glFlush();
}
```

Ejemplo de luz ambiente

```
#include<gl/glut.h>
void iniciar()
{
    GLfloat color_difuso_y_ambiente[]={0.8,0.8,0.8,1.0};
    GLfloat color_especular[]={0.1,0.5,0.8,1.0};
    GLfloat brillo_especular[]={2.0};
    glClearColor (0.0,0.0,0.0,0.0);
    glMaterialfv(GL_FRONT, GL_AMBIENT_AND_DIFFUSE, color_difuso_y_ambiente);
    glMaterialfv(GL_FRONT, GL_SPECULAR, color_especular);
    glMaterialfv(GL_FRONT, GL_SHININESS, brillo_especular);
    glEnable(GL_LIGHTING);
    glEnable(GL_LIGHT0);
}
```

```
void dibuja(void)
{
    GLfloat luz_posicion[]={0.0,0.2,-0.8,1.0};
    GLfloat luz_ambiente_ninguna[]={0.0,0.0,0.0,1.0};
    GLfloat luz_ambiente_suave[]={0.2,0.2,0.2,1.0};
    GLfloat luz_ambiente_fuerte[]={0.6,0.2,0.2,1.0};
    glClear(GL_COLOR_BUFFER_BIT);

    glPopMatrix();
    glPushMatrix();
    glTranslatef(0.7,0.0,0.0);
    glLightModelfv(GL_LIGHT_MODEL_AMBIENT, luz_ambiente_fuerte);
    glLightfv(GL_LIGHT0, GL_POSITION, luz_posicion);
    glutSolidSphere(0.3,25,25);
    glPopMatrix();
    glFlush();
}
```

```
int main (int argc, char**argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB );
    glutInitWindowPosition(100,100);
    glutInitWindowSize(500,500);
    glutCreateWindow("Luz Ambiente");
    iniciar();
    glutDisplayFunc(dibuja);
    glutMainLoop();
    return 0;
}
```

¡GRACIAS!

